

Analysis and Design Manual: Coding Tutor

Georgios Soulantikas

April 2026

Contents

1	Pedagogical Design	3
1.1	Educational Objectives and Target Audience	3
1.2	Curriculum Structure and Organization	3
1.3	Self-Assessment and Active Participation	4
1.4	Adaptive Learning and Individualization	4
1.4.1	The Hint Trigger Mechanism	4
2	Technical Implementation	5
2.1	System Architecture Overview	5
2.2	Content Management and Data Pipeline	5
2.2.1	The Synchronization Engine	5
2.2.2	Data Fetching and Rendering	6
2.3	Sandboxed Execution Engine	6
2.3.1	Container Isolation	6
2.4	Assessment Logic and Telemetry	6
2.4.1	Trial vs. Validation Workflows	6
2.4.2	Data Persistence and Progress Tracking	7
2.5	Database Schema	7
2.5.1	User Data Model	7
2.6	Security and Authentication	7
3	User Interface and Experience Design	8
3.1	Design Philosophy	8
3.2	Dashboard and Information Architecture	8
3.2.1	Visualizing Progression	8
3.2.2	Performance Metrics	9
3.3	The Dual-Pane Learning Environment	9
3.4	The Interactive Workspace	10

3.4.1	Editor and Console Distinction	11
3.4.2	Action Buttons and Visual Weight	11
3.4.3	Post-Success Interface Updates	11
3.5	Typography for Code Legibility	11
3.6	Non-Intrusive Feedback Mechanisms	12
3.6.1	Pedagogical and Success Feedback	12
3.6.2	Authentication Error Handling	12
4	Self-Assessment	13

Chapter 1

Pedagogical Design

1.1 Educational Objectives and Target Audience

The main goal of the "Coding Tutor" application is to provide a clear and solid introduction to Python programming. The curriculum is designed specifically for beginners, using a step-by-step approach where each unit builds directly on the logic learned in previous lessons.

1.2 Curriculum Structure and Organization

The course content is divided into six units, defined in the system's configuration file (`units.json`). This structure gradually increases in difficulty, guiding the user from basic syntax to more complex concepts:

- **Unit 1: Python Foundations** – Introduction to variables and printing.
- **Unit 2: Decision Making** – Logical operators and conditional statements (if/else).
- **Unit 3: Loops & Automation** – Repeating actions and the `range()` function.
- **Unit 4: Data Collections** – Managing grouped data using lists.

- **Unit 5: Reusable Tools** – Organizing code with custom functions.
- **Unit 6: Building Projects** – Combining learned skills through practical exercises.

1.3 Self-Assessment and Active Participation

Unlike passive video tutorials, the app keeps students engaged with interactive "missions." Each lesson includes a coding challenge where the student's code is checked against the correct answer (`expectedOutput`) using the Piston API. This provides the instant feedback that is so important for self-paced learning.

1.4 Adaptive Learning and Individualization

To personalize the learning experience, the application includes a smart hint system that adapts to how well the student is doing.

1.4.1 The Hint Trigger Mechanism

The system tracks how the user's code performs. If a student's output doesn't match the `expectedOutput`, the app counts this as a failed attempt (f). The hint system follows this simple logic:

$$H(f) = \begin{cases} \text{Hidden} & \text{if } f < 2 \\ \text{Visible} & \text{if } f \geq 2 \end{cases}$$

Waiting until the second failed attempt encourages students to try debugging their own code first. This promotes a healthy "trial-and-error" learning style before stepping in to help.

Chapter 2

Technical Implementation

2.1 System Architecture Overview

The application is built using a modern, separated architecture with three main parts:

- **Frontend Layer:** A React application written in TypeScript, using Tailwind CSS to look great on any screen size.
- **Execution Layer:** A Dockerized Piston engine that safely runs the students' Python code in an isolated environment.
- **Persistence Layer:** Google Firebase for saving progress in real-time and securely managing user logins.

2.2 Content Management and Data Pipeline

One of the best technical features of the platform is how easily new lessons can be added, keeping the course content completely separate from the app's code.

2.2.1 The Synchronization Engine

Lessons are written as standard Markdown files with a YAML header. However, the React app doesn't load these files directly. Instead, a Node.js script (`sync_lessons.js`) processes them, grabs the important data (like

`pretypedCode` and `expectedOutput`), and uploads it straight to the Firebase database. This keeps the web app lightweight and fast.

2.2.2 Data Fetching and Rendering

When a user opens a lesson, the `LessonPage` component asks Firebase for the data using the lesson's `id`. Once the text is downloaded, a custom `MarkdownRenderer` turns it into the interactive interface the student sees.

To make reading easier for beginners, this renderer uses CSS to turn off "font ligatures." This means that symbols like `<=` or `!=` stay as two separate characters (just like they look on a keyboard) rather than merging into a single complex symbol.

2.3 Sandboxed Execution Engine

Running code submitted by users can be dangerous. To keep the server safe, the platform uses Piston, a fast and secure code execution engine running inside Docker.

2.3.1 Container Isolation

The `docker-compose.yml` file sets up the Piston service. Piston uses `nsjail` to create temporary, locked-down environments for every piece of code. This prevents the code from accessing the main network or reading private files. Additionally, a startup script ensures Python 3.12.0 is installed the moment the server turns on, so students never have to wait for their code to run.

2.4 Assessment Logic and Telemetry

The `LessonPage.tsx` component handles the main learning loop through two different actions: Trial and Validation.

2.4.1 Trial vs. Validation Workflows

- **Trial (`runCode`):** The user's code is sent to Piston, and the result (`stdout` or `stderr`) is printed on the screen. This lets students experi-

ment and test things safely without recording any "failed" attempts in the database.

- **Validation** (`submitCode`): This is the actual grading system. The result is cleaned up using `.trim()` and strictly compared against the database's `expectedOutput`.

2.4.2 Data Persistence and Progress Tracking

When a student successfully submits a lesson, the system updates their profile in Firebase. It adds the lesson's ID to their `completedLessons` list. This reliable tracking is what powers the real-time statistics shown on the student's Dashboard.

2.5 Database Schema

To provide rich and helpful statistics, the system uses a flexible NoSQL structure in Firebase Firestore.

2.5.1 User Data Model

Each user profile (`UserData`) keeps a detailed log of their learning journey:

- **Completed Lessons:** A list of the lesson IDs the student has successfully finished.
- **Attempt Tracking:** A counter that remembers how many times a user tried to solve a specific lesson before passing.

2.6 Security and Authentication

User logins are handled securely by Firebase Auth. The app uses an `AuthContext` provider to ensure that only logged-in students can view the lessons or their personal dashboards.

Chapter 3

User Interface and Experience Design

3.1 Design Philosophy

The visual design of the "Coding Tutor" application is rooted in minimalism and cognitive load reduction. For a beginner, learning to program is already a complex mental task. Therefore, the interface was designed to be as unobtrusive as possible, removing unnecessary visual clutter and utilizing a clean, high-contrast aesthetic that keeps the user's focus entirely on the educational content.

3.2 Dashboard and Information Architecture

Upon logging in, the student is presented with the Dashboard. The primary user experience (UX) goal of this screen is to provide immediate, encouraging feedback on the student's overall progress.

3.2.1 Visualizing Progression

The curriculum is broken down into a grid of distinct "Unit" cards. Rather than relying solely on text or simple checklists, the interface utilizes bold, circular progress indicators for each unit. This visual approach allows the student to instantly grasp their standing in the course at a glance. Partially

filled rings act as a subtle psychological motivator, encouraging the student to complete the remaining lessons to "close the circle."

3.2.2 Performance Metrics

At the top of the dashboard, a prominent statistics banner displays the user's total code submissions and average attempts per lesson. Presenting these metrics in large, clear typography transforms standard database tracking into a gamified element. It shifts the user's goal from simply completing lessons to improving their coding accuracy over time.

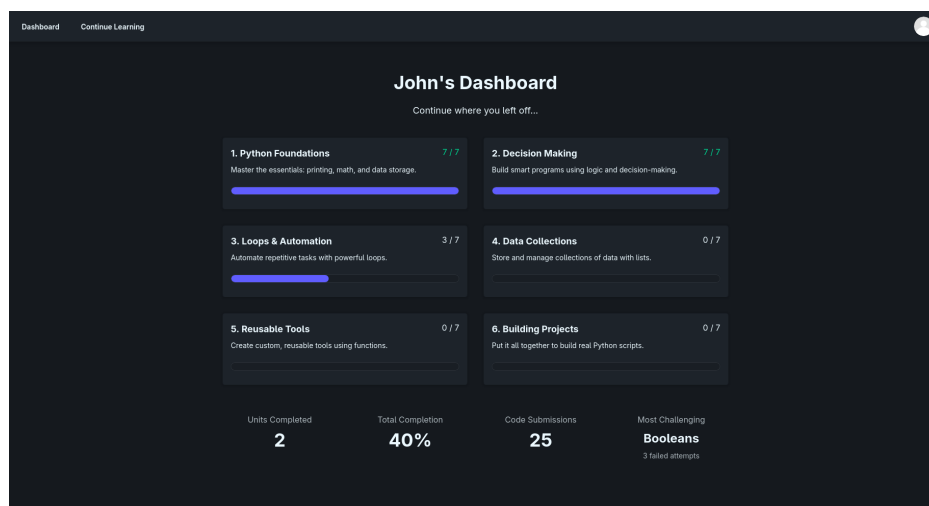


Figure 3.1: The Dashboard interface utilizing card layouts and circular progress indicators.

3.3 The Dual-Pane Learning Environment

The most critical screen in the application is the Lesson Page. The primary design challenge here was presenting both theoretical instruction and an interactive coding workspace without overwhelming the user.

To solve this, the desktop interface employs a strict dual-pane, side-by-side layout.

- **The Theory Pane (Left):** This area is dedicated to reading. It uses a light, clean background with generous margins and distinct headings to make the markdown content easily scannable.
- **The Interactive Workspace (Right):** This area houses the code editor and console.

This side-by-side design is a deliberate pedagogical choice. It eliminates the need for the student to constantly scroll up and down between the instructions and their code, preventing the loss of context and maintaining their flow state.

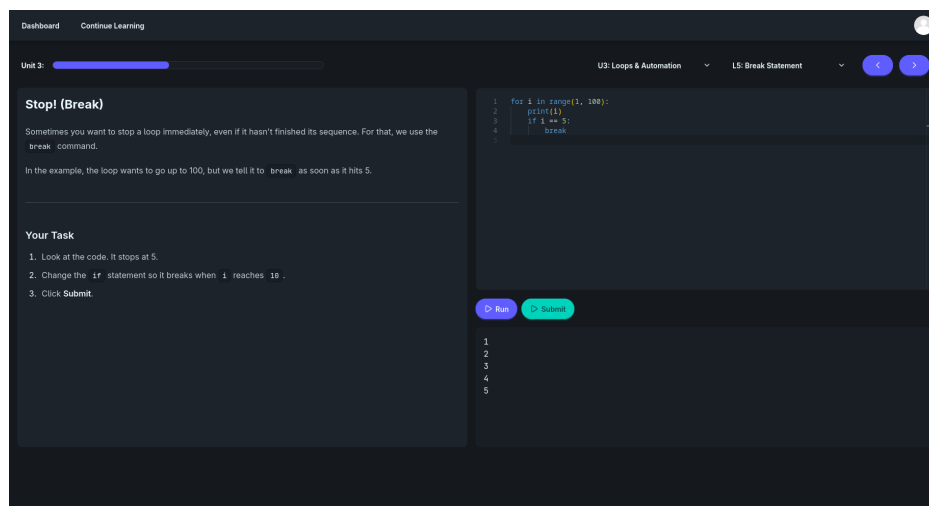


Figure 3.2: The split-screen Lesson Page separating theory (left) from practice (right).

3.4 The Interactive Workspace

The right side of the Lesson Page was designed to look and feel like a professional programming environment, while remaining approachable for absolute beginners.

3.4.1 Editor and Console Distinction

The code editor features a clean, light interface with prominent line numbers to help students track their logic. Directly below the editor is the output console. The console deliberately uses a dark background and monospace font. This high-contrast visual separation clearly distinguishes the code the student writes from the response the computer gives, mimicking the look of a real developer terminal.

3.4.2 Action Buttons and Visual Weight

The interactive actions are governed by two distinct buttons, designed with clear visual hierarchy:

- **Run Code (Secondary Action):** Styled with a muted, outlined appearance. This signals that clicking it is a safe, low-stakes action meant for testing and debugging.
- **Submit (Primary Action):** Styled with a solid, bold color block. This high visual weight draws the user's eye and clearly indicates the final, formal action of having their code graded.

3.4.3 Post-Success Interface Updates

A crucial aspect of the UX is how the interface adapts once a student successfully completes a lesson:

- **Subtle State Shifts:** The "Submit" button gracefully shifts its thematic color (from the accent color to the primary color) to indicate the completed state.
- **Navigation Unlock:** The completed state is passed to the sticky top navigation bar, updating the unit's progress bar and unlocking the path to the next lesson.

3.5 Typography for Code Legibility

Reading code requires different typographic considerations than reading standard text. The interface ensures that the student can easily spot small syntax errors.

To aid in the cognitive parsing of raw code, the interactive editor integrates robust syntax highlighting tailored specifically for Python. By assigning distinct, high-contrast colors to different semantic elements—such as keywords, strings, variables, and comments—the interface allows beginners to visually dissect the structure of their logic. This immediate visual feedback helps students quickly identify missing quotation marks or unclosed parentheses before even attempting to run their code.

3.6 Non-Intrusive Feedback Mechanisms

In educational software, the way the system communicates success or failure drastically impacts the user’s emotional state. The application explicitly avoids native browser alerts, which are jarring and disruptive. Instead, it utilizes a custom “Toast” notification system that slides smoothly onto the screen.

3.6.1 Pedagogical and Success Feedback

During the lessons, these toasts provide instant feedback. When a user submits an incorrect answer, a gentle error toast encourages them to try again. When they succeed, the system displays a highly visible success toast containing playful, context-aware messages (e.g., “You took the `else` path to victory!”). This specialized feedback rewards the student while reinforcing the programming concepts they just learned.

3.6.2 Authentication Error Handling

The same elegant feedback system is applied to the login and registration flows. Instead of exposing raw database errors (such as `auth/invalid-credential`), the application intercepts these codes and translates them into friendly, human-readable toast notifications (e.g., “The email or password you entered is incorrect”). This ensures that even at the very first touchpoint of the application, the user feels guided and supported.

Chapter 4

Self-Assessment

Lorem ipsum...